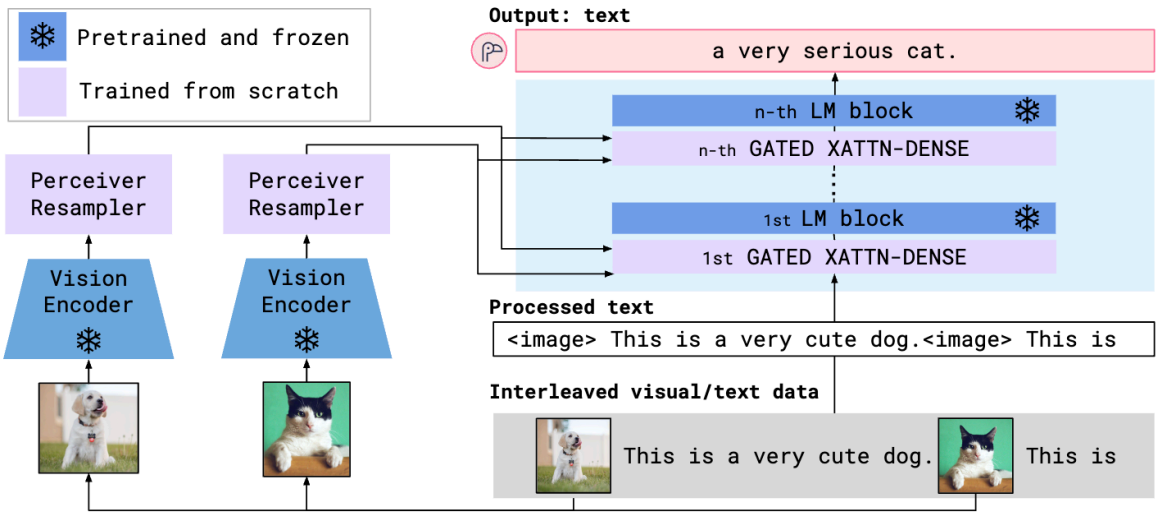
# Flamingo(2022, DeepMind)

## 模型结构




## 输入输出格式

输入 text interleaved with images 的形式，输出是 text

### Vision Encoder

视觉编码器采用 NFNet（NormalizerFree ResNet），作者先在图文对数据上采用 CLIP 的方式对 NFNet 进行预训练。在 Flamingo 训练过程中，Vision Encoder 权重固定·

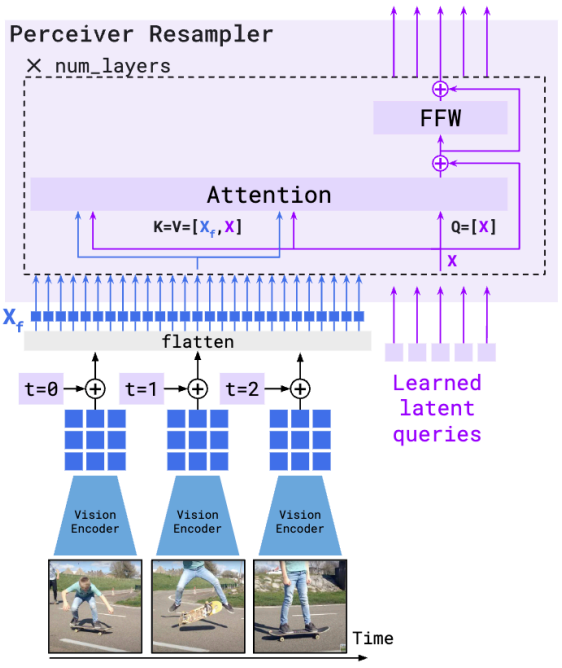
如果视觉端输入是视频，则按照 1 fps 进行采样后将 N 帧进行视觉特征提取（若是图片输入，则 N=1）。

### LLM

1.4B, 7B, and 70B Chinchilla。并且这三个量级的 LLM 分别对应 Flamingo-3B, Flamingo-9B, Flamingo-80B

### Perceiver Resampler

Perceiver Resampler（感知重采样器）可以接收任意多个的视频帧（如果是图像，可以视作是单帧视频），经过 Vision Encoder 提取特征，加上 positional embedding 之后全部展平，得到一个视觉 token 序列 $X_f$, $X_f$ 的序列长度是任意的。还会有一个可学习的固定长度的 latent query 序列 $X$ 。$X_f$ 和 $X$ 同时作为输入，送入到 Perceiver Resampler 中计算交叉注意力，注意力的 K 和 V 是 $X_f$ 和 $X$ 拼接而成，而 Q 就是 $X$ 。在经过多层 Attention + FFW（其实就是 FFN）处理之后，输出固定长度（文中是 64）的 latent query，作为视觉表征。




```
def perceiver_resampler(
    x_f,  # The [T, S, d] visual features (T=time, S=space)
    time_embeddings,  # The [T, 1, d] time pos embeddings.
    x,  # R learned latents of shape [R, d]
    num_layers,  # Number of layers
):
  """The Perceiver Resampler model."""

  # Add the time position embeddings and flatten.
  x_f = x_f + time_embeddings
  x_f = flatten(x_f)  # [T, S, d] -> [T * S, d]
  # Apply the Perceiver Resampler layers.
  for i in range(num_layers):
    # Attention.
    x = x + attention_i(q=x, kv=concat([x_f, x]))
    # Feed forward.
    x = x + ffw_i(x)
  return x
```

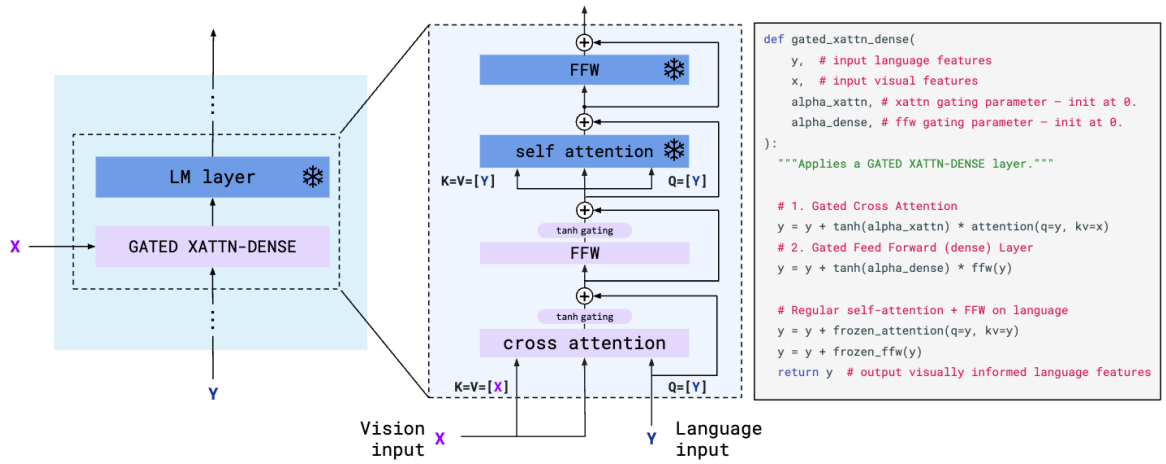
注意这里的 positional embedding 只是 time embedding，即标识图片的时序信息，即图片来自哪一帧，这是因为单张图片内的空间位置信息在 vision encoder 中已经处理过了。

代码块

```
def perceiver_resampler(
        x_f, # The [T, S, d] visual features (T=time, S=space)
        time_embeddings, # The [T, 1, d] time pos embeddings.
        x, # R learned latents of shape [R, d]
        num_layers, # Number of layers
):
        """The Perceiver Resampler model."""
        # Add the time position embeddings and flatten.
        x_f = x_f + time_embeddings
        x_f = flatten(x_f) # [T, S, d] -> [T * S, d]
        # Apply the Perceiver Resampler layers.
        for i in range(num_layers):
                # Attention.
                x = x + attention_i(q=x, kv=concat([x_f, x]))
                # Feed forward.
                x = x + ffw_i(x)
        return x
```

## Gated Xattn-Dense Layers

Flamingo 将固定长度的视觉 query 注入到语言模型的方法称为 Gated xattn-dense，其详细结构示意图及伪代码如下图所示。具体来说，在预训练好的 LM 的各层交替地插入一些随机初始化的交叉注意力层。所谓 gated 门控，在每一新插入的层之后的残差链接之前的输出乘上一个可学习的 tanh gating，即 $\tanh(\alpha) = \frac{e^{\alpha}-e^{-\alpha}}{e^{\alpha}+e^{-\alpha}}$，其中 $\alpha$ 是一个可学习的标量值，初始值为 $0$，从而保证初始化时的输出与原 LM 一致。



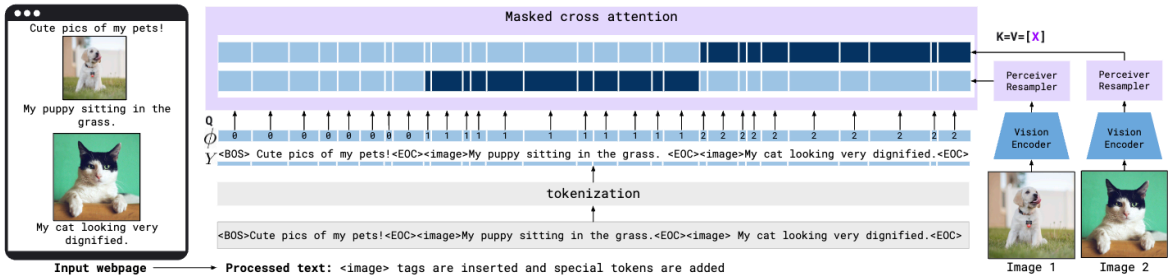

代码块

```
def gated_xattn_dense(
        y, # input language features
        x, # input visual features
        alpha_xattn, # xattn gating parameter – init at 0.
        alpha_dense, # ffw gating parameter – init at 0.
):
        """Applies a GATED XATTN-DENSE layer."""
        # 1. Gated Cross Attention
        y = y + tanh(alpha_xattn) * attention(q=y, kv=x)
        # 2. Gated Feed Forward (dense) Layer
        y = y + tanh(alpha_dense) * ffw(y)
        # Regular self-attention + FFW on language
        y = y + frozen_attention(q=y, kv=y)
        y = y + frozen_ffw(y)
        return y # output visually informed language features
```

# 训练

Flamingo 的训练数据有三类：图文交错数据集、图像文本对数据集、视频文本对数据集。其中图文交错数据集是 Flamingo 数据的重点，其多模态 in-context learning（few-shot learning）的能力可以说主要就是来自图文交错数据。作者收集了一个大规模图文交错数据集 M3W ，通过解析 HTML 获取并标记图片在文本中的位置。

Flamingo 的训练还有一个关键点，就是如何处理图文交错数据。首先每张图片的特征用 vision encoder + resampler 处理得到固定长度的 token 序列，在文本序列中用 `<image>` 这个 tag 来标记出图像的位置。值得一提的是，每个文本 token 只会 attend 到之前最近的一张图片，而不是所有之前的图片，之前的其他图片会被 mask 掉。



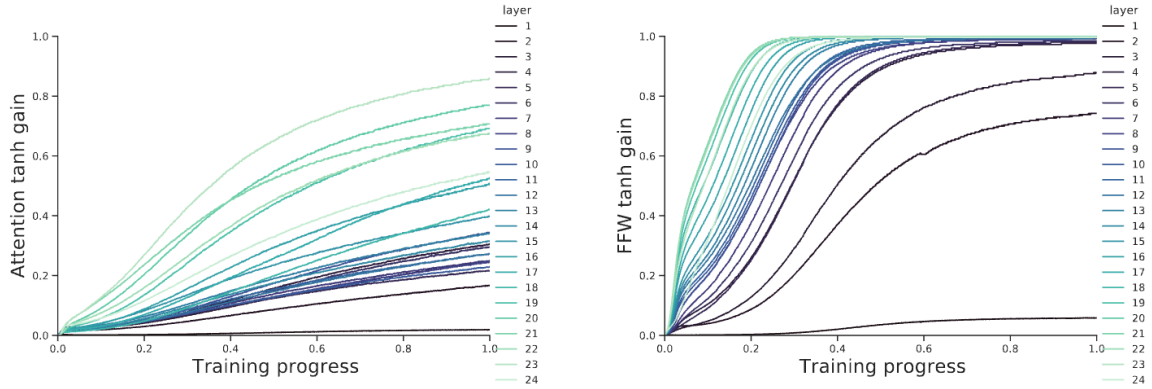

# 实验

模型在各个 benchmark 的表现：

| Method | VQAV2 | | COCO | VATEX | VizWiz | | MSRVTTQA | VisDial | | YouCook2 | TextVQA | | HatefulMemes |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | test-dev | test-std | test | test | test-dev | test-std | test | valid | test-std | valid | valid | test-std | test seen |
| 32 shots | 67.6 | - | 113.8 | 65.1 | 49.8 | - | 31.0 | 56.8 | - | 86.8 | 36.0 | - | 70.0 |
| Fine-tuned | **82.0** | **82.1** | 138.1 | **84.2** | **65.7** | **65.4** | **47.4** | 61.8 | 59.7 | 118.6 | **57.1** | 54.1 | **86.6** |
| SotA | 81.3† [133] | 81.3† [133] | **149.6**† [119] | 81.4† [153] | 57.2† [65] | 60.6† [65] | 46.8 [51] | **75.2** [79] | **75.4**† [123] | **138.7** [132] | 54.7 [137] | **73.7** [84] | 84.6† [152] |

Table 2: **Comparison to SotA when fine-tuning *Flamingo*.** We fine-tune *Flamingo* on all nine tasks where *Flamingo* does not achieve SotA with few-shot learning. *Flamingo* sets a new SotA on five of them, outperfoming methods (marked with †) that use tricks such as model ensembling or domain-specific metric optimisation (e.g., CIDEr optimisation).

Gated Xattn-Dense 可训练参数 $\alpha$ 随层深、训练 step 发生的变化。发现训练步数增加，层深增加，视觉语义融合加大。




(a) Attention tanh gating

(b) FFW tanh gating.

Figure 6: Evolution of the absolute value of the tanh gating at different layers of *Flamingo*-3B.

再来看一下消融实验：

模型结构上来说，gating、resampler、cross attention layer 都是必须的

数据上来说 M3W 的缺失影响巨大

| | Ablated setting | *Flamingo*-3B original value | Changed value | Param. count ↓ | Step time ↓ | COCO CIDEr↑ | OKVQA top1↑ | VQAv2 top1↑ | MSVDQA top1↑ | VATEX CIDEr↑ | Overall score↑ |
|---|---|---|---|---|---|---|---|---|---|---|---|
| | | ***Flamingo*-3B model** | | 3.2B | 1.74s | 86.5 | 42.1 | 55.8 | 36.3 | 53.4 | **70.7** |
| **(i)** | Training data | All data | w/o Video-Text pairs | 3.2B | 1.42s | 84.2 | 43.0 | 53.9 | 34.5 | 46.0 | 67.3 |
| | | | w/o Image-Text pairs | 3.2B | 0.95s | 66.3 | 39.2 | 51.6 | 32.0 | 41.6 | 60.9 |
| | | | Image-Text pairs→ LAION | 3.2B | 1.74s | 79.5 | 41.4 | 53.5 | 33.9 | 47.6 | 66.4 |
| | | | w/o M3W | 3.2B | 1.02s | 54.1 | 36.5 | 52.7 | 31.4 | 23.5 | 53.4 |
| **(ii)** | Optimisation | Accumulation | Round Robin | 3.2B | 1.68s | 76.1 | 39.8 | 52.1 | 33.2 | 40.8 | 62.9 |
| **(iii)** | Tanh gating | ✓ | ✗ | 3.2B | 1.74s | 78.4 | 40.5 | 52.9 | 35.9 | 47.5 | 66.5 |
| **(iv)** | Cross-attention architecture | GATED XATTN-DENSE | VANILLA XATTN | 2.4B | 1.16s | 80.6 | 41.5 | 53.4 | 32.9 | 50.7 | 66.9 |
| | | | GRAFTING | 3.3B | 1.74s | 79.2 | 36.1 | 50.8 | 32.2 | 47.8 | 63.1 |
| **(v)** | Cross-attention frequency | Every | Single in middle | 2.0B | 0.87s | 71.5 | 38.1 | 50.2 | 29.1 | 42.3 | 59.8 |
| | | | Every 4th | 2.3B | 1.02s | 82.3 | 42.7 | 55.1 | 34.6 | 50.8 | 68.8 |
| | | | Every 2nd | 2.6B | 1.24s | 83.7 | 41.0 | 55.8 | 34.5 | 49.7 | 68.2 |
| **(vi)** | Resampler | Perceiver | MLP | 3.2B | 1.85s | 78.6 | 42.2 | 54.7 | 35.2 | 44.7 | 66.6 |
| | | | Transformer | 3.2B | 1.81s | 83.2 | 41.7 | 55.6 | 31.5 | 48.3 | 66.7 |
| **(vii)** | Vision encoder | NFNet-F6 | CLIP ViT-L/14 | 3.1B | 1.58s | 76.5 | 41.6 | 53.4 | 33.2 | 44.5 | 64.9 |
| | | | NFNet-F0 | 2.9B | 1.45s | 73.8 | 40.5 | 52.8 | 31.1 | 42.9 | 62.7 |
| **(viii)** | Freezing LM | ✓ | ✗ (random init) | 3.2B | 2.42s | 74.8 | 31.5 | 45.6 | 26.9 | 50.1 | 57.8 |
| | | | ✗ (pretrained) | 3.2B | 2.42s | 81.2 | 33.7 | 47.4 | 31.0 | 53.9 | 62.7 |

Table 3: **Ablation studies.** Each row should be compared to the baseline Flamingo run (top row). Step time measures the time spent to perform gradient updates on all training datasets.